MACHINE LEARNING

LAB RECORD

ITA0614 | EXPERIMENT 3

EXPERIMENT STATEMENT

Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

STUDENT NAME	REGISTER NUMBER
P H Devesh	**1911260236**
SUBJECT CODE	**SUBJECT NAME**
ITA0614	**Machine Learning**

Five program demonstrations • Executed Python examples • Verified output

SUB-PROGRAM 1

ID3 from Scratch: Play Tennis Classification

Problem Statement / Program Title

ID3 from Scratch: Play Tennis Classification.

Aim

Implement ID3 using entropy and information gain, then classify a new weather sample.

Algorithm

1. Store the categorical training examples and target labels.
2. Calculate entropy of the target labels.
3. For each remaining attribute, calculate weighted child entropy and information gain.
4. Choose the attribute with the highest information gain as the decision node.
5. Recursively build branches until a node is pure or no attributes remain.
6. Traverse the resulting tree to classify the new sample.

Program

```
from collections import Counter
from math import log2

data = [
    {"Outlook":"Sunny", "Humidity":"High", "Wind":"Weak", "Play":"No"},
    {"Outlook":"Sunny", "Humidity":"High", "Wind":"Strong", "Play":"No"},
    {"Outlook":"Overcast", "Humidity":"High", "Wind":"Weak", "Play":"Yes"},
    {"Outlook":"Rain", "Humidity":"High", "Wind":"Weak", "Play":"Yes"},
    {"Outlook":"Rain", "Humidity":"Normal", "Wind":"Weak", "Play":"Yes"},
    {"Outlook":"Rain", "Humidity":"Normal", "Wind":"Strong", "Play":"No"},
    {"Outlook":"Overcast", "Humidity":"Normal", "Wind":"Strong", "Play":"Yes"},
    {"Outlook":"Sunny", "Humidity":"High", "Wind":"Weak", "Play":"No"},
    {"Outlook":"Sunny", "Humidity":"Normal", "Wind":"Weak", "Play":"Yes"},
    {"Outlook":"Rain", "Humidity":"Normal", "Wind":"Weak", "Play":"Yes"},
    {"Outlook":"Sunny", "Humidity":"Normal", "Wind":"Strong", "Play":"Yes"},
    {"Outlook":"Overcast", "Humidity":"High", "Wind":"Strong", "Play":"Yes"},
    {"Outlook":"Overcast", "Humidity":"Normal", "Wind":"Weak", "Play":"Yes"},
    {"Outlook":"Rain", "Humidity":"High", "Wind":"Strong", "Play":"No"},
]
target = "Play"

def entropy(rows):
    counts = Counter(row[target] for row in rows)
    total = len(rows)
    return -sum((n / total) * log2(n / total) for n in counts.values())

def majority(rows):
    return Counter(row[target] for row in rows).most_common(1)[0][0]

def id3(rows, attributes):
    labels = {row[target] for row in rows}
    if len(labels) == 1:
        return next(iter(labels))
    if not attributes:
        return majority(rows)
    base = entropy(rows)
    best = max(attributes, key=lambda a: base - sum(
        len(group) / len(rows) * entropy(group)
        for value in {r[a] for r in rows}
        for group in [[r for r in rows if r[a] == value]]
    ))
    tree = {best: {}}
    for value in sorted({row[best] for row in rows}):
        subset = [row for row in rows if row[best] == value]
        tree[best][value] = id3(subset, [a for a in attributes if a != best])
    return tree

def predict(tree, sample, fallback="Yes"):
    if not isinstance(tree, dict):
        return tree
    attribute = next(iter(tree))
    branch = tree[attribute].get(sample.get(attribute))
    if branch is None:
        return fallback
    return predict(branch, sample, fallback)

tree = id3(data, ["Outlook", "Humidity", "Wind"])
sample = {"Outlook":"Sunny", "Humidity":"Normal", "Wind":"Strong"}
print("Training examples:", len(data))
print("Root attribute:", next(iter(tree)))
print("New sample:", sample)
print("Predicted class:", predict(tree, sample))
```

Input / Dataset

Categorical Play Tennis training set: 14 examples with Outlook, Humidity, Wind, and target Play. The records are embedded directly in the program.

Representative records: Outlook=Sunny, Humidity=High, Wind=Weak → Play=No; Outlook=Overcast, Humidity=Normal, Wind=Weak → Play=Yes.

Output

```
Training examples: 14
Root attribute: Outlook
New sample: {'Outlook': 'Sunny', 'Humidity': 'Normal', 'Wind': 'Strong'}
Predicted class: Yes
```

Result

The program built an ID3 tree using entropy and information gain, then classified the supplied weather sample.

SUB-PROGRAM 2

ID3 from Scratch: Restaurant Decision

Problem Statement / Program Title

ID3 from Scratch: Restaurant Decision.

Aim

Build a small ID3 tree for deciding whether to eat at a restaurant based on categorical attributes.

Algorithm

1. Define the restaurant dataset with target labels Yes or No.
2. Use Shannon entropy to measure impurity in the target.
3. Compute information gain for each available feature.
4. Split on the feature with the highest gain and repeat recursively.
5. Stop when the examples are pure or no features remain.
6. Classify the supplied new restaurant situation using the learned tree.

Program

```
from collections import Counter
from math import log2

rows = [
    {"Hungry":"Yes","Wait":"Long","Price":"High","Eat":"No"},
    {"Hungry":"Yes","Wait":"Short","Price":"High","Eat":"Yes"},
    {"Hungry":"No","Wait":"Short","Price":"Low","Eat":"No"},
    {"Hungry":"Yes","Wait":"Short","Price":"Low","Eat":"Yes"},
    {"Hungry":"No","Wait":"Long","Price":"High","Eat":"No"},
    {"Hungry":"Yes","Wait":"Long","Price":"Low","Eat":"Yes"},
    {"Hungry":"No","Wait":"Short","Price":"High","Eat":"No"},
    {"Hungry":"Yes","Wait":"Long","Price":"High","Eat":"No"},
]
target = "Eat"

def entropy(examples):
    counts = Counter(x[target] for x in examples)
    n = len(examples)
    return -sum((c/n) * log2(c/n) for c in counts.values())

def build_id3(examples, features):
    labels = {x[target] for x in examples}
    if len(labels) == 1:
        return labels.pop()
    if not features:
        return Counter(x[target] for x in examples).most_common(1)[0][0]
    base = entropy(examples)
    gains = {}
    for feature in features:
        values = {x[feature] for x in examples}
        remainder = 0
        for value in values:
            subset = [x for x in examples if x[feature] == value]
            remainder += len(subset)/len(examples) * entropy(subset)
        gains[feature] = base - remainder
    best = max(gains, key=gains.get)
    node = {best:{}}
    for value in sorted({x[best] for x in examples}):
        subset = [x for x in examples if x[best] == value]
        node[best][value] = build_id3(subset, [f for f in features if f != best])
    return node

def classify(tree, item):
    if not isinstance(tree, dict):
        return tree
    feature = next(iter(tree))
    value = item.get(feature)
    if value not in tree[feature]:
        return "Unknown"
    return classify(tree[feature][value], item)

tree = build_id3(rows, ["Hungry", "Wait", "Price"])
new_case = {"Hungry":"Yes", "Wait":"Short", "Price":"Low"}
print("Training records:", len(rows))
print("Learned tree:", tree)
print("New case:", new_case)
print("Decision:", classify(tree, new_case))
```

Input / Dataset

Eight illustrative restaurant records with Hungry, Wait, Price, and target Eat. The small dataset is embedded directly in the program.

Example: Hungry=Yes, Wait=Short, Price=Low → Eat=Yes.

Output

```
Training records: 8
Learned tree: {'Hungry': {'No': 'No', 'Yes': {'Wait': {'Long': {'Price': {'High': 'No', 'Low': 'Yes'}}, 'Short': 'Yes'
New case: {'Hungry': 'Yes', 'Wait': 'Short', 'Price': 'Low'}
Decision: Yes
```

Result

The program learned a categorical decision tree from the restaurant examples and returned a decision for the new case.

SUB-PROGRAM 3

ID3 with scikit-learn: Weather Classification

Problem Statement / Program Title

ID3 with scikit-learn: Weather Classification.

Aim

Train an entropy-based decision tree using scikit-learn and evaluate its classification on a held-out test set.

Algorithm

1. Create a labelled categorical weather dataset.
2. Encode categorical values as integer features.
3. Separate training and test records deterministically.
4. Train DecisionTreeClassifier with criterion='entropy'.
5. Predict the test labels and report accuracy and a confusion matrix.
6. Classify a new weather sample using the trained model.

Program

```
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

X_text = [
    ["Sunny","Hot","High"], ["Sunny","Mild","High"],
    ["Overcast","Hot","High"], ["Rain","Mild","High"],
    ["Rain","Cool","Normal"], ["Rain","Cool","Normal"],
    ["Overcast","Cool","Normal"], ["Sunny","Cool","Normal"],
    ["Sunny","Mild","Normal"], ["Rain","Mild","Normal"],
    ["Sunny","Hot","Normal"], ["Overcast","Mild","High"],
]
y_text = ["No","No","Yes","Yes","Yes","No","Yes","Yes","Yes","Yes","Yes","Yes"]

encoder = OrdinalEncoder()
X = encoder.fit_transform(X_text)
labels = LabelEncoder()
y = labels.fit_transform(y_text)

# Fixed split: first 9 rows for training, last 3 for testing.
model = DecisionTreeClassifier(criterion="entropy", random_state=0)
model.fit(X[:9], y[:9])
pred = model.predict(X[9:])
actual = y[9:]

new_weather = encoder.transform([["Sunny","Mild","High"]])
new_prediction = labels.inverse_transform(model.predict(new_weather))[0]

print("Training samples:", len(X[:9]))
print("Test samples:", len(X[9:]))
print("Actual test labels:", labels.inverse_transform(actual).tolist())
print("Predicted test labels:", labels.inverse_transform(pred).tolist())
print("Accuracy:", round(accuracy_score(actual, pred), 3))
print("Confusion matrix (label order No, Yes):")
print(confusion_matrix(actual, pred, labels=[labels.transform(["No"])[0],
labels.transform(["Yes"])[0]]))
print("New sample [Sunny, Mild, High]:", new_prediction)
```

Input / Dataset

Twelve categorical weather records with Outlook, Temperature, Humidity, and target Yes/No. First 9 rows are used for training and last 3 for testing; this fixed split is intentional.

New sample: Sunny, Mild, High. Ordinal encoding is fitted to the feature categories in the full small example so the categories are consistently represented.

Output

```
Training samples: 9
Test samples: 3
Actual test labels: ['Yes', 'Yes', 'Yes']
Predicted test labels: ['No', 'Yes', 'Yes']
Accuracy: 0.667
Confusion matrix (label order No, Yes):
[[0 0]
 [1 2]]
New sample [Sunny, Mild, High]: No
```

Result

The entropy-based decision tree produced predictions for the held-out test records and classified a new weather sample. Accuracy reflects this small fixed split only.

SUB-PROGRAM 4

Student Pass/Fail Prediction using ID3

Problem Statement / Program Title

Student Pass/Fail Prediction using ID3.

Aim

Apply an entropy-based decision tree to predict whether a student passes using study and attendance categories.

Algorithm

1. Prepare a small illustrative student dataset with categorical features and Pass/Fail labels.
2. Encode feature categories consistently.
3. Train a decision tree using entropy as the split criterion.
4. Predict the outcome for a new student profile.
5. Display the predicted class and training accuracy.
6. State that the dataset is synthetic and the result is a demonstration, not a real academic assessment.

Program

```
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Synthetic teaching dataset; not real student records.
features = [
    ["High","Good","Regular"], ["High","Poor","Regular"],
    ["Medium","Good","Regular"], ["Low","Good","Regular"],
    ["Low","Poor","Irregular"], ["Medium","Poor","Irregular"],
    ["High","Good","Irregular"], ["Medium","Good","Irregular"],
    ["Low","Good","Irregular"], ["High","Poor","Irregular"],
    ["Medium","Poor","Regular"], ["Low","Poor","Regular"],
]
outcomes =
["Pass","Pass","Pass","Pass","Fail","Fail","Pass","Pass","Fail","Fail","Fail","Fail"]

feature_encoder = OrdinalEncoder()
X = feature_encoder.fit_transform(features)
target_encoder = LabelEncoder()
y = target_encoder.fit_transform(outcomes)

model = DecisionTreeClassifier(criterion="entropy", random_state=7)
model.fit(X, y)

new_student = [["High","Good","Regular"]]
encoded_student = feature_encoder.transform(new_student)
prediction = target_encoder.inverse_transform(model.predict(encoded_student))[0]

print("Dataset type: Synthetic illustrative data")
print("Number of records:", len(features))
print("Training accuracy:", round(accuracy_score(y, model.predict(X)), 3))
print("New student [Study=High, Attendance=Good, Assignments=Regular]")
print("Predicted outcome:", prediction)
```

Input / Dataset

Synthetic illustrative dataset: 12 student profiles using Study level, Attendance, and Assignment completion categories, labelled Pass/Fail. These are not real student records.

Output

```
Dataset type: Synthetic illustrative data
Number of records: 12
Training accuracy: 1.0
New student [Study=High, Attendance=Good, Assignments=Regular]
Predicted outcome: Pass
```

Result

The entropy-based classifier predicted a pass/fail label for the supplied synthetic student profile. This is only a programming demonstration, not a reliable real-world student prediction.

SUB-PROGRAM 5

Fruit Type Classification using ID3

Problem Statement / Program Title

Fruit Type Classification using ID3.

Aim

Classify a fruit from categorical colour, size, and surface features using an entropy-based decision tree.

Algorithm

1. Create a small synthetic dataset describing fruit examples.
2. Encode categorical input features and target classes.
3. Fit a decision tree using information gain (entropy).
4. Predict the class of a new fruit with unseen combination of feature values.
5. Print the sample, predicted fruit, and training accuracy.
6. Interpret the result only as a demonstration on the supplied synthetic dataset.

Program

```
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Synthetic illustrative fruit examples.
fruit_features = [
    ["Red","Small","Smooth"], ["Red","Small","Smooth"],
    ["Green","Small","Smooth"], ["Yellow","Medium","Smooth"],
    ["Yellow","Large","Rough"], ["Green","Large","Rough"],
    ["Orange","Medium","Rough"], ["Orange","Small","Rough"],
    ["Red","Large","Smooth"], ["Yellow","Medium","Rough"],
    ["Green","Medium","Smooth"], ["Orange","Large","Rough"],
]
fruit_labels = [
    "Apple","Apple","Apple","Banana","Banana","Watermelon",
    "Orange","Orange","Apple","Banana","Apple","Orange"
]

feature_encoder = OrdinalEncoder()
X = feature_encoder.fit_transform(fruit_features)
target_encoder = LabelEncoder()
y = target_encoder.fit_transform(fruit_labels)

model = DecisionTreeClassifier(criterion="entropy", random_state=3)
model.fit(X, y)

new_fruit = [["Orange","Medium","Rough"]]
prediction = target_encoder.inverse_transform(
    model.predict(feature_encoder.transform(new_fruit))
)[0]

print("Dataset type: Synthetic illustrative data")
print("Training examples:", len(fruit_features))
print("Training accuracy:", round(accuracy_score(y, model.predict(X)), 3))
print("New fruit [Orange, Medium, Rough]")
print("Predicted fruit:", prediction)
```

Input / Dataset

Synthetic illustrative dataset: 12 fruit examples described by Colour, Size, and Surface, labelled by fruit type. These are not measured or real-world records.

Output

```
Dataset type: Synthetic illustrative data
Training examples: 12
Training accuracy: 1.0
New fruit [Orange, Medium, Rough]
Predicted fruit: Orange
```

Result

The entropy-based classifier predicted a fruit label for the supplied synthetic feature combination. The result is limited by the small illustrative dataset.